

GDA Doctrine Automation Spec

Last updated: 2026-05-08
Status: Draft agreed in working session
Scope: Sprint doctrine capture, draft persistence, finalize-to-GitHub publishing

Purpose

This specification defines the automation pattern for keeping GDA's long-lived operating doctrine current without requiring Shawn to manually edit documents. The design aligns with the current GDA architecture: React is the cockpit, n8n is the workflow engine, Postgres is the system of record for important business state, GitHub is the canonical source of truth for long-lived code and documentation, and gist is only a mirror.[file:5][file:3]

The immediate goal is to automate creation and publication of sprint notes, Book of Truths updates, and doctrine deltas from real repo changes and verified platform state, while preserving human approval only for risky operational actions.[file:5]

Agreed decisions

Topic	Decision
Publish model	PR merge drafts updates; "Finalize Sprint" publishes canon.[file:5]
Draft storage	Hybrid: Postgres is source of truth; GitHub draft markdown files are visibility mirrors.[file:3][file:5]
Publish gate	Finalization is blocked unless required checks are green.[file:5]
Canonical docs store	GitHub repository is canonical for long-lived documentation.[file:5][file:4]
Workflow engine	n8n remains the engine for this automation; do not rebuild elsewhere unnecessarily.[file:5]
Mirror behavior	gist is a downstream mirror only, not the deepest source of truth.[file:5][file:3]

Operating model

Draft phase

A merged PR triggers an n8n workflow that classifies the change by sprint, component, and doctrine impact, then writes a normalized draft record into Postgres. The same workflow regenerates a human-visible GitHub draft file such as docs/sprints/S-206-draft.md, so the current draft state is visible in the repo while Postgres remains the source of truth.[file:3][file:5]

The draft workflow should derive at least four views from each merged change: current live state, planned state, recommended next state, and safety-lane impact. This matches the

operator-style reporting model already defined for the Space and keeps implementation status separated from planning status.[file:5]

Finalize phase

A separate "Finalize Sprint" n8n workflow is manually triggered with a `sprintId`. Before it publishes any canon documents, it verifies required checks: React build/CI status, QA Center read-only and dry-run checks for critical workflows, and consistency guards from Master Build such as preserving frozen workflows and lane discipline.[file:5][file:4][file:3]

If the checks pass, the workflow reads all draft doctrine rows for the sprint from Postgres, builds final markdown artifacts, updates canonical GitHub docs, and optionally updates the gist mirror after GitHub succeeds. If checks fail, finalization is blocked and the workflow emits a plain-English reason instead of publishing stale or misleading documentation.[file:5][file:3]

Proposed repository layout

```
/docs/
  Book_of_Truths.md
  /sprints/
    S-206-draft.md
    S-206.md
  /decisions/
    approvals-queue-draft.md
    approvals-queue.md
```

This keeps doctrine discoverable in GitHub while preserving a clear separation between draft and canonical documents. It also matches the operating rule that GitHub, not gist, should remain the canonical home for long-lived documentation.[file:5]

Postgres source-of-truth model

Postgres should store normalized doctrine draft records, not only rendered markdown blobs, because structured fields are easier to validate, merge, diff, and reuse across multiple documents. This follows the broader architectural rule that important business state should live in Postgres rather than be scattered across workflow memory.[file:5][file:3]

Recommended table: doctrine_drafts

Column	Type	Purpose
id	UUID PK	Stable draft record ID.
sprint_id	text	Sprint identifier such as S-206.
component	text	Affected surface, e.g. Approvals Queue, QA Center.
doc_type	text	book_of_truths, sprint_notes, decision_log, master_build_note.

Column	Type	Purpose
title	text	Short doctrine item title.
status	text	draft, finalized, superseded, blocked.
source_pr_number	integer null	Originating GitHub PR number.
source_pr_url	text null	PR URL for traceability.
source_commit_sha	text null	Merge commit SHA.
workflow_name	text null	Producing workflow name if generated from automation.
safety_lane_impact	text null	read-only, dry-run, test-row, approval-required, unknown, or none.[file:5]
current_live_state_md	text	Verified current-state markdown.
planned_state_md	text null	Planned but not live markdown.
recommended_next_state_md	text null	Recommended next-step markdown.
decision_md	text null	Exact doctrine / policy language.
checks_required_json	jsonb	Required checks before publish.
checks_result_json	jsonb null	Last gate evaluation result.
rendered_markdown	text	Rendered draft section used for GitHub mirror.
last_updated_at	timestampz	Audit timestamp.
finalized_at	timestampz null	Set when promoted to canon.
finalized_commit_sha	text null	Git commit that published canon.
created_by	text	Usually automation identity.
updated_by	text	Automation identity or approved operator.

Recommended supporting table: doctrine_publish_runs

Column	Type	Purpose
id	UUID PK	Publish run ID.
sprint_id	text	Target sprint.
trigger_type	text	pr-merge, finalize-sprint, manual-rebuild.
status	text	running, passed, failed, blocked.
check_summary_json	jsonb	All gate results in one object.
error_message	text null	Plain-English failure reason.
github_commit_sha	text null	Resulting docs commit if published.

Column	Type	Purpose
started_at	timestampz	Run start time.
finished_at	timestampz null	Run end time.

n8n workflow design

Workflow A: PR merge to doctrine draft

Trigger: GitHub PR merged to main.

Goal: update Postgres draft state and GitHub draft mirrors automatically.

Core steps:

1. Receive GitHub webhook for merged PR.
2. Parse PR metadata, labels, files changed, sprint ID, and component scope.
3. Classify doctrine impact into current live state, planned state, recommended next state, and safety-lane impact.[file:5]
4. Upsert doctrine_drafts rows in Postgres.
5. Render sprint-level markdown draft from Postgres rows.
6. Commit or update docs/sprints/S-XXX-draft.md and any decision draft files in GitHub.
7. Record run details in doctrine_publish_runs.

Workflow B: Finalize Sprint

Trigger: manual execution with sprintId.

Goal: promote validated drafts into canonical GitHub docs.

Required checks before publish:

- GitHub/CI build for current main is green.[file:4]
- Required React smoke or route checks are green where applicable.[file:5]
- QA Center read-only and dry-run checks for required workflows are green.[file:5]
- Master Build guards confirm frozen workflows remain untouched and no critical write-capable workflow is missing a safety lane.[file:3][file:5]

Core steps:

1. Load sprint draft rows from Postgres.
2. Run gate checks and build a structured pass/fail summary.
3. If any required check fails, stop and emit a plain-English blocked reason.[file:5]
4. Build final markdown for docs/sprints/S-XXX.md.
5. Merge approved doctrine sections into docs/Book_of_Truths.md.
6. Optionally update docs/decisions/*.md for durable decisions.

7. Commit to GitHub using bot credentials in the automation environment, not in the browser or chat sandbox.[file:5][file:4]
8. Mark relevant Postgres draft rows finalized and store resulting commit SHA.
9. Optionally trigger gist mirror update after GitHub succeeds.[file:3][file:5]

Approval and safety boundary

This automation should freely update documentation and draft doctrine because documentation publication is not one of the risky action categories called out for approval-required execution. Human approval remains reserved for risky operational actions such as sends, deletes, deploys, paid research, and real production writes.[file:5]

Even so, docs automation should still obey platform safety norms:

- No secrets in browser code; all GitHub writes happen server-side via n8n or backend credentials.[file:5][file:4]
- Failures should be visible in a plain-English status surface, ideally QA Center or an adjacent platform health view.[file:5]
- GitHub stays canonical; gist remains a mirror only.[file:5]

Recommended first implementation order

1. Create Postgres tables doctrine_drafts and doctrine_publish_runs.[file:3]
2. Create a GitHub bot token stored in n8n credentials; do not expose it to the frontend.[file:5][file:4]
3. Build Workflow A for PR merge to draft sync.
4. Write docs/sprints/S-XXX-draft.md automatically from Postgres rows.
5. Build Workflow B for gated finalize-to-canon publishing.
6. Add plain-English failure reporting for blocked finalize runs.
7. Add optional gist mirroring only after GitHub publishing is working.[file:3][file:5]

Repository-ready next artifact

The most useful first repository artifact is this spec itself, committed under docs/automation/gda-doctrine-automation-spec.md, followed by a small SQL migration for the doctrine tables and an n8n workflow export once implemented. That sequence creates a durable written contract before implementation drift starts.[file:5][file:3][file:4]